Operations Research Project Report

Scheduling of a Flexible Flow Line with Limited Buffers and Bypass

Chapter 16.3 — Pinedo, *Scheduling: Theory, Algorithms, and Systems*

June 23, 2026

Abstract

This report analyzes the scheduling problem described in Section 16.3 of Pinedo's *Scheduling: Theory, Algorithms, and Systems* [6]: the cyclic scheduling of a flexible flow line with limited buffers and job bypass, originally addressed at IBM via the Flexible Flow Line Loading (FFLL) algorithm [4]. We (1) formalize the problem and its Graham three-field classification, (2) derive and critically assess a mixed-integer programming (MIP) formulation, (3) describe the three-phase FFLL heuristic and apply it to two original numerical examples, and (4) propose an Iterated Greedy with Local Search (IG-LS) algorithm inspired by the iterated greedy framework of Ruiz and Stützle [7], implement it, and benchmark it against FFLL on ten randomly generated instances. IG-LS reduces the average optimality gap from 33.8% (FFLL) to 15.1%, reaching the lower bound on two of the ten instances.

Contents

1 Problem Statement and Graham Classification

1.1 Problem Description

The problem considers a **flexible flow line (FFL)** – a production environment consisting of multiple stages arranged in series, where each stage contains one or more machines operating in parallel. The specific context in Section 16.3 of Pinedo [6] is the assembly of printed circuit boards (PCBs), an environment for which the Flexible Flow Line Loading (FFLL) algorithm was originally designed at IBM [4].

The environment has the following defining characteristics:

Jobs Each job represents a batch of relatively small identical items (e.g., PCBs). A set of jobs that together represent one full day's production mix is called a *Minimum Part Set* (MPS). The manufacturing process is repetitive, so the MPS cycles through the system continuously.

Processing Each job must be processed at every stage on exactly one machine. Not all machines at a stage are necessarily identical – some may handle only specific job types.

Bypass A job may not require processing at every stage. If a job skips a stage, the material handling system transports it past that stage. This is modeled by setting the processing time of that job at the bypassed stage to zero, $p_{ij} = 0$.

Limited buffers Between stages, buffer space is finite. If a buffer is full, the material handling system halts – or, if recirculation is possible, the job bypasses the stage and recirculates.

Cyclic schedule Because production is repetitive, the objective is a *cyclic schedule*: the best ordering and timing of jobs within one MPS cycle, repeated indefinitely.

1.2 Objectives and Alternative Objective Functions

The FFLL algorithm pursues two objectives simultaneously [6]:

- **Primary objective – maximize throughput.** In cyclic scheduling this is equivalent to minimizing the cycle time of one MPS. The cycle time cannot be smaller than the workload of the bottleneck machine, so the algorithm seeks to approach this lower bound.
- **Secondary objective – minimize work-in-process (WIP).** Since buffer space is limited, reducing WIP lowers the probability of buffer overflow and the resulting blocking.

Alternative objective functions that could be substituted for $C_{\max}$ include:

- Minimizing weighted tardiness $\sum w_j T_j$ – relevant when jobs carry due dates and differing priorities.
- Minimizing total/average flow time $\sum C_j$ – reducing the average time a job spends in the system.
- Minimizing the number of tardy jobs $\sum U_j$ – relevant in customer-facing settings.
- Minimizing maximum lateness $L_{\max}$ – relevant when due-date violations carry hard consequences.

1.3 Graham Notation

We use the standard three-field notation $\alpha \mid \beta \mid \gamma$ of Graham et al. [1], adopting Pinedo's machine-environment symbol for this specific class of problems. Pinedo denotes the flexible flow shop by **FFc**, where c is the number of stages in series, each stage being a bank of *parallel machines* [6]. The classification is therefore:

$$\boxed{FFc \mid bypass,\ b_i < \infty \mid C_{\max}}$$

Field	Symbol	Meaning
α (machine environment)	FFc	Flexible flow shop: c stages in series, each stage a bank of parallel machines [6]
β (constraints)	*bypass*	Jobs may skip a stage (zero processing time there)
	$b_i < \infty$	Finite buffer capacity between stages
γ (objective)	$C_{\max}$	Minimize makespan / cycle time of the MPS

Table 1: Graham three-field classification of the flexible flow line scheduling problem.

Remark. FFc generalizes both the classical flow shop (Fm, single machine per stage) and the parallel-machine environment (Pm, a single stage); it reduces to Fm when every stage has exactly one machine. Adding *bypass* means routing is no longer uniform across jobs, and finite buffers ($b_i < \infty$) introduce *blocking*, a phenomenon absent from most classical scheduling theory. Even the two-machine flow shop with blocking is known to be NP-hard [3], and the combination of parallel machines, bypass, and finite buffers studied here is at least as hard. This NP-hardness is the central motivation for the heuristic (rather than exact) solution approach of Section 3.

2 MIP Formulation and Critical Assessment

Pinedo's chapter does not present an explicit mixed-integer program; we construct one here that mirrors the three decisions made sequentially by the FFLL algorithm: machine assignment, job sequencing, and timing.

2.1 Indices, Parameters and Decision Variables

Indices and parameters.

- n: number of jobs in the MPS, indexed $j = 1, \ldots, n$.
- m: total number of machines, indexed $i = 1, \ldots, m$.
- S: number of stages, indexed $s = 1, \ldots, S$.
- M_s: set of machines belonging to stage s.
- p_{ij}: processing time of job j on machine i (0 if job j bypasses the stage containing machine i).
- b_s: buffer capacity between stage s and stage $s+1$.
- L: a sufficiently large constant (big-M).

Decision variables.

Variable	Domain	Meaning
x_{ij}	$\{0,1\}$	1 if job j is assigned to machine i
$y_{ijj'}$	$\{0,1\}$	1 if job j precedes job j' on machine i
C_{ij}	$\mathbb{R}_{\geq 0}$	Completion time of job j on machine i
T	$\mathbb{R}_{\geq 0}$	Cycle time (makespan of one MPS)

2.2 MIP Model

$$\text{minimize} \quad T \tag{1}$$

$$\text{s.t.} \quad \sum_{i \in M_s} x_{ij} = 1 \qquad \forall j,\ \forall s \tag{C1}$$

$$C_{ij} \geq C_{ij'} + p_{ij} x_{ij} - L(1 - y_{ijj'}) \qquad \forall i,\ \forall j \neq j' \tag{C2a}$$

$$C_{ij'} \geq C_{ij} + p_{ij'} x_{ij'} - L\, y_{ijj'} \qquad \forall i,\ \forall j \neq j' \tag{C2b}$$

$$C_{ij} \geq C_{i'j} + p_{ij} x_{ij} \qquad \forall j,\ s,\ i \in M_s,\ i' \in M_{s-1} \tag{C3}$$

$$\Big|\{\, j : C_{i'j} \leq t < C_{ij} \,\}\Big| \leq b_s \qquad \forall s,\ \forall t \tag{C4}$$

$$T \geq C_{ij} \qquad \forall i, j : x_{ij} = 1 \tag{C5}$$

$$T \geq W_i = \sum_{j=1}^{n} p_{ij} \qquad \forall i \tag{C6}$$

$$x_{ij},\, y_{ijj'} \in \{0,1\}, \quad C_{ij}, T \geq 0 \tag{C7}$$

(C1) assigns every job to exactly one machine per stage – bypassed stages satisfy this trivially since $p_{ij} = 0$ for all $i \in M_s$. **(C2a–C2b)** are the classical disjunctive constraints preventing two jobs from overlapping on the same machine. **(C3)** enforces the series structure of the flow line: a job cannot start processing at stage s before completing stage $s-1$. **(C4)** restricts the number of jobs simultaneously resident in the buffer between consecutive stages to the buffer capacity b_s; in practice this requires either time-indexed binary variables or an auxiliary counting formulation, since it is not linear as written. **(C5)** defines the cycle time as the latest completion time across all jobs and machines. **(C6)** is a valid inequality stating that the cycle time cannot be smaller than the total workload of any single machine – this tightens the LP relaxation considerably.

2.3 Model Size and Complexity

Component	Count
Binary variables x_{ij}	$m \times n$
Binary sequencing variables $y_{ijj'}$	$m \times n \times (n-1)$
Continuous variables C_{ij}	$m \times n$
Disjunctive constraints (C2a)–(C2b)	$O(mn^2)$
Buffer constraints (C4)	$O(S \cdot T_{\text{horizon}})$

Table 2: Growth of the MIP's size with problem dimensions.

Even for a modest instance with $m = 5$ machines and $n = 10$ jobs, the number of binary sequencing variables already reaches 450, and the disjunctive constraints yield a notoriously weak LP relaxation – a well-documented issue for job-shop-style MIPs [6]. The buffer constraint (C4) is the most problematic component, as a faithful linearization requires time-indexed binary variables, exploding the model size for any non-trivial planning horizon.

2.4 Critical Assessment of the MIP

Strengths.

- Provides an exact representation; guarantees global optimality given sufficient solver time.
- The bottleneck lower bound (C6) tightens the relaxation and accelerates branch-and-bound.
- Flexible: alternative objectives (WIP, weighted tardiness) can be substituted directly into (1).
- Useful as an exact benchmark for evaluating heuristic solution quality on small instances.

Weaknesses.

- The disjunctive constraints (C2a)–(C2b) are the classical weakness of job-shop MIPs, producing enormous branch-and-bound trees even for small instances.
- The buffer constraints (C4) are not linear as stated and require an expensive time-indexed reformulation.
- The cyclic nature of the schedule – where the end of one MPS cycle feeds directly into the start of the next – is not captured by the static formulation above without additional wrap-around constraints.
- The bypass feature, while trivial to encode via $p_{ij} = 0$, interacts subtly with the buffer constraints, since bypassing jobs still occupy the material handling system.
- The model does not scale: industrial PCB assembly lines may involve dozens of stages and hundreds of job types per MPS, far beyond the practical reach of exact MIP solvers.

Overall verdict. The MIP is theoretically sound and a useful small-instance benchmark, but it is not a practical solution method for industrial-scale instances of this problem – precisely the motivation behind IBM's development of the FFLL heuristic [4].

3 The FFLL Algorithm

3.1 Algorithm Description

The Flexible Flow Line Loading (FFLL) algorithm [4, 6] solves the problem in three sequential phases, each addressing one of the three core decisions: *who* processes *what*, *in what order*, and *when*.

Phase 1 — Machine Allocation (LPT Heuristic)

Each job is assigned to one machine per stage so that workloads across parallel machines at that stage are as balanced as possible – balance directly determines the bottleneck workload and hence the achievable cycle time. The Longest Processing Time (LPT) heuristic [2] is applied independently at every stage with multiple machines:

1. Sort all jobs in decreasing order of processing time at that stage.
2. Assign each job, one at a time, to the currently least-loaded machine at that stage.
3. Repeat until all jobs are assigned.

Phase 2 — Sequencing (Dynamic Balancing Heuristic)

Let n be the number of jobs and m the number of machines. Define

$$W_i = \sum_{j=1}^{n} p_{ij} \qquad \text{workload of machine } i,$$

$$W = \sum_{i=1}^{m} W_i \qquad \text{total system workload,}$$

$$p_k = \sum_{i=1}^{m} p_{ik} \qquad \text{system-wide workload of job } k,$$

$$o_{ij} = p_{ij} - p_j \frac{W_i}{W} \qquad \text{overload contribution of job } j \text{ to machine } i,$$

$$O_{ij} = \sum_{k \in S_j} o_{ik} \qquad \text{cumulative overload on machine } i \text{ after position } j,$$

where S_j is the set of jobs loaded up to and including position j in the sequence. The Dynamic Balancing heuristic greedily minimizes

$$\sum_{i=1}^{m} \sum_{j=1}^{n} \max(O_{ij}, 0) \tag{2}$$

by, at each position, selecting from the remaining unscheduled jobs the one that minimizes the resulting total positive cumulative overload.

Phase 3 — Timing of Releases (Bottleneck Anchoring)

1. Identify the bottleneck machine $i^* = \arg\max_i W_i$; the cycle time lower bound is $T^* = W_{i^*}$.
2. Release all jobs into the system as rapidly as possible and simulate the resulting schedule.
3. Fix the start/completion times on the bottleneck machine – no idle time is permitted there.
4. For machines *upstream* of the bottleneck, delay processing as much as possible without altering job sequences, reducing WIP.
5. For machines *downstream* of the bottleneck, process jobs as early as possible, again without altering sequences.

3.2 Two Original Application Examples

The following two examples are constructed independently of the textbook and differ from Pinedo's Example 16.3.1.

Example A — Two Stages, Four Jobs, No Bypass

Stage 1 has two parallel machines; Stage 2 has a single machine; the MPS contains four jobs.

	Job 1	Job 2	Job 3	Job 4
Stage 1	5	8	3	6
Stage 2	4	2	7	5

Table 3: Stage processing times p'_{kj} for Example A.

Phase 1 (LPT at Stage 1). Sorting descending: Job 2 (8), Job 4 (6), Job 1 (5), Job 3 (3). Assignment: Job 2→M1 (load 8), Job 4→M2 (load 6), Job 1→M2 (load 11), Job 3→M1 (load 11). Both Stage 1 machines balance perfectly at load 11; Machine 3 (Stage 2) processes all jobs, $W_3 = 18$. **Bottleneck: Machine 3,** $T^* = 18$**.**

Phase 2 (Dynamic Balancing). With $W = (11, 11, 18)$, $\sum W = 40$, and $p_k = (9, 10, 10, 11)$, evaluating o_{ij} and applying the greedy rule position-by-position yields the sequence $\mathbf{1} \to \mathbf{3} \to \mathbf{2} \to \mathbf{4}$.

Phase 3 (Timing). The bottleneck machine processes jobs back-to-back in this order; the achieved cycle time is $T = 18$, matching the lower bound exactly (optimal for this instance).

Example B — Three Stages, Five Jobs, with Bypass

Stage 1 has one machine; Stage 2 has two parallel machines; Stage 3 has one machine; the MPS contains five jobs, with Jobs 2 and 5 bypassing Stage 2.

	Job 1	Job 2	Job 3	Job 4	Job 5
Stage 1	3	5	2	4	6
Stage 2	4	0	6	3	0
Stage 3	2	3	1	4	2

Table 4: Stage processing times p'_{kj} for Example B (Jobs 2, 5 bypass Stage 2).

Phase 1 (LPT at Stage 2). Only Jobs 1, 3, 4 require Stage 2. Sorting descending: Job 3 (6), Job 1 (4), Job 4 (3). Assignment: Job 3→M2 (load 6), Job 1→M3 (load 4), Job 4→M3 (load 7). The resulting workloads are $W = (20, 6, 7, 12)$, $\sum W = 45$. **Bottleneck: Machine 1 (Stage 1),** $T^* = 20$**.**

Phase 2 (Dynamic Balancing). With $p_k = (9, 8, 9, 11, 8)$, evaluating o_{ij} and applying the greedy rule position-by-position yields the sequence $\mathbf{2} \to \mathbf{1} \to \mathbf{3} \to \mathbf{5} \to \mathbf{4}$.

Phase 3 (Timing). The bottleneck Machine 1 processes jobs back-to-back in this order; Jobs 2 and 5 bypass Stage 2 directly. The achieved cycle time is $T = 20$, again matching the lower bound exactly.

3.3 Critical Assessment of the FFLL Heuristic

Strengths.

- Computationally efficient: Phase 1 runs in $O(n \log n)$ per stage, Phase 2 in $O(n^2 m)$, and Phase 3 in linear time – tractable for the hundreds of job types found in industrial MPS instances.
- The three-phase decomposition mirrors the natural decision hierarchy: allocation must precede sequencing (workloads must be known first), and sequencing must precede timing.
- The Dynamic Balancing objective (2) has a clear theoretical motivation: minimizing burst loading directly reduces buffer occupancy and blocking probability.
- Bypass is handled naturally, since bypassed stages simply contribute zero to all o_{ij} terms.
- Extensive practical experiments reported by Pinedo confirm FFLL's value as a scheduling tool in real assembly environments [6].

Limitations.

- The three phases are solved *independently and sequentially* with no feedback loop – a poor Phase 1 allocation cannot be corrected by Phases 2 or 3.

- LPT does not guarantee optimal balance; Pinedo himself notes this explicitly for Stage 3 of Example 16.3.1.
- The Dynamic Balancing heuristic is *greedy with no lookahead*: a locally optimal choice at position j may preclude a substantially better global sequence.
- WIP minimization is addressed only indirectly, through Phase 3's timing adjustments, with no explicit guarantee against blocking under tight buffers.
- The algorithm assumes a fixed, fully known MPS and offers no mechanism for incremental re-optimization under dynamic arrivals or uncertain processing times.

4 Improved Algorithm: Iterated Greedy with Local Search (IG-LS)

4.1 Design Rationale

The principal weakness identified in Section 3.3 is that FFLL's Phase 2 is a *non-backtracking greedy* procedure: once a job is placed, it remains fixed regardless of how later choices unfold. This is precisely the failure mode that the *iterated greedy* (IG) metaheuristic framework of Ruiz and Stützle [7] was designed to address for permutation flow shop scheduling. Since its introduction, IG has been successfully adapted to numerous structurally similar flow shop variants – including blocking flow shops, no-idle flow shops, and sequence-dependent setup-time problems – frequently outperforming more elaborate metaheuristics while remaining simple to implement [5, 7]. This motivates adapting the same destruction–reconstruction principle to the FFLL sequencing phase.

We propose **IG-LS**, which retains FFLL's Phase 1 (LPT allocation) and Phase 3 (bottleneck timing) unchanged, and replaces Phase 2 with three layers:

1. **Warm start.** Initialize with the FFLL Dynamic Balancing sequence.
2. **Local search (2-opt).** Exhaustively evaluate pairwise job swaps; accept any swap that reduces the *actual simulated makespan*; repeat until no improving swap remains.
3. **Iterated perturbation.** Remove a random contiguous block of jobs and reinsert it at a random position (destruction–reconstruction, following Ruiz and Stützle [7]); re-apply local search; accept the new solution only if it improves the incumbent makespan. Repeat for a fixed iteration budget.

Crucially, IG-LS optimizes the *true simulated makespan* rather than the overload proxy (2) used by Dynamic Balancing – a more direct and unbiased optimization signal.

4.2 Pseudocode

4.3 Experimental Results on Ten Random Instances

Ten instances were generated with deliberately varied characteristics – ranging from small 4-job/2-stage problems to larger 12-job/5-stage problems, with varying bypass probabilities and numbers of machines per stage (full generator and parameters in the accompanying source code, `ffll_igls.py`). The lower bound (LB) reported is the bottleneck machine workload after LPT allocation (constraint (C6)).

IG-LS improved *every one* of the ten instances. On Instances 3 and 4 it reached the theoretical lower bound, proving optimality. The largest gains occur on Instances 5, 6, and 9, where FFLL's greedy sequencing committed to particularly poor early choices that IG-LS's perturbation and local search were able to correct. Across all ten instances, the average gap to the lower bound fell from 33.8% (FFLL) to 15.1% (IG-LS), while all runs completed in well under one second.

Algorithm 1 IG-LS: Iterated Greedy with Local Search

```
procedure IG-LS(instance, n_iter = 150, block_size = 2)
    alloc ← LPT_Allocate(instance)                    ▷ Phase 1, identical to FFLL
    seq ← DynamicBalancing(alloc)                     ▷ warm start
    (seq, best_ms) ← LocalSearch2opt(seq, alloc)
    for k = 1 to n_iter do
        seq′ ← Perturb(seq, block_size)               ▷ remove & reinsert block
        (seq′, ms′) ← LocalSearch2opt(seq′, alloc)
        if ms′ < best_ms then
            seq ← seq′;   best_ms ← ms′
        end if
    end for
    cycle_time ← BottleneckTiming(seq, alloc)         ▷ Phase 3, identical to FFLL
    return seq, cycle_time, best_ms
end procedure
```

#	Jobs	Stg	LB	FFLL	FFLL gap	IG-LS	IG-LS gap	Improv.
1	4	2	9.0	13.00	44.4%	12.00	33.3%	7.7%
2	5	2	23.0	26.00	13.0%	25.00	8.7%	3.8%
3	6	3	32.0	35.00	9.4%	**32.00**	**0.0%**	8.6%
4	7	3	38.0	42.00	10.5%	**38.00**	**0.0%**	9.5%
5	8	4	21.0	43.00	104.8%	34.00	61.9%	20.9%
6	8	3	34.0	49.00	44.1%	35.00	2.9%	28.6%
7	10	4	46.0	56.00	21.7%	50.00	8.7%	10.7%
8	10	5	49.0	56.00	14.3%	54.00	10.2%	3.6%
9	12	4	66.0	97.00	47.0%	72.00	9.1%	25.8%
10	12	5	63.0	81.00	28.6%	73.00	15.9%	9.9%
				Average	**33.8%**		**15.1%**	**12.9%**

Table 5: FFLL vs. IG-LS on ten randomly generated instances. Gaps are relative to the bottleneck lower bound (LB). Bold entries indicate proven-optimal solutions (gap = 0%).

4.4 Critical Assessment of IG-LS

Strengths over FFLL.

- More than halves the average optimality gap (33.8% → 15.1%).
- Reaches proven optimality on two of ten instances.
- Improves consistently across all tested instance types, not just isolated cases.
- Optimizes actual makespan directly, rather than an overload proxy.
- Remains computationally cheap (sub-second per instance at this scale), consistent with reports in the IG literature that the framework is simple yet highly competitive against more elaborate metaheuristics [7].

Remaining limitations.

- The gap to the lower bound remains substantial on some instances (e.g., Instance 5: 61.9%); note the bound itself may be loose, so the true optimality gap could be smaller than reported.
- The purely greedy acceptance criterion (accept only strict improvements) may miss solutions reachable only via temporary uphill moves; a simulated-annealing-style acceptance criterion, as used in several IG variants [7], could help.

- The perturbation block size is fixed at 2; an adaptive schedule could better balance exploration and exploitation.
- Like FFLL, the three phases remain solved independently; a fully integrated metaheuristic spanning allocation and sequencing could yield further gains at higher computational cost.
- Scalability beyond the tested range (up to 12 jobs) is untested; the $O(n^2)$ per-iteration cost of 2-opt local search will eventually dominate for much larger MPS instances.

The complete Python implementation (FFLL, IG-LS, instance generator, and experiment runner) accompanies this report as `ffll_igls.py`.

References

[1] R. L. Graham, E. L. Lawler, J. K. Lenstra, and A. H. G. Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics*, 5:287–326, 1979. doi: 10.1016/S0167-5060(08)70356-X.

[2] Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969. doi: 10.1137/0117039.

[3] Nicholas G. Hall and Chelliah Sriskandarajah. A survey of machine scheduling problems with blocking and no-wait in process. *Operations Research*, 44(3):510–525, 1996. doi: 10.1287/opre.44.3.510.

[4] IBM Corporation. The Flexible Flow Line Loading (FFLL) algorithm for printed circuit board assembly scheduling, 1990. As described in Pinedo, *Scheduling: Theory, Algorithms, and Systems*, Section 16.3.

[5] Quan-Ke Pan and Rubén Ruiz. An effective iterated greedy algorithm for the mixed no-idle permutation flowshop scheduling problem. *Omega*, 44:41–50, 2014. doi: 10.1016/j.omega.2013.10.002.

[6] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, New York, 5th edition, 2016.

[7] Rubén Ruiz and Thomas Stützle. A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research*, 177(3):2033–2049, 2007. doi: 10.1016/j.ejor.2005.12.009.